

Deep Learning for Solving Dynamic Models

Session 3a: Neural architecture search, random search and Hyperband

Simon Scheidegger

HEC, University of Lausanne

University of Padova

September 23, 2026

Roadmap of This Lecture

I. Motivation

- ▶ Why architecture matters
- ▶ The hyperparameter space
- ▶ Limits of manual tuning

II. Search Methods

- ▶ Grid vs. random search
- ▶ Bayesian optimization
- ▶ Hyperband / successive halving
- ▶ Method comparison

Hands-on, in `day1/code/03_nas_loss_balancing/`: `03_01_Grid_vs_Random_Search.ipynb` (where to put a fixed number of trials)

`03_02_NAS_RandomSearch_Hyperband.ipynb` (how to spend a fixed number of epochs, and the same search on a DEQN)

III. Implementing the Search

- ▶ Defining a search space
- ▶ Random search and successive halving from scratch

IV. Example & Wrap-up

- ▶ Genz-function approximation
- ▶ The same search on a DEQN
- ▶ Best practices

Learning objectives

By the end of this lecture you should be able to:

- ▶ Measure, rather than assume, which hyperparameters move the error on a given problem
- ▶ State when random search beats grid search at a fixed budget, and when it does not
- ▶ Explain what Bayesian optimization and successive halving add to random search, and at what cost
- ▶ Implement random search and successive halving in plain Python, with a train / validation / test split and an exact epoch budget
- ▶ Run the same search on a deep equilibrium net, where the score is an Euler-equation error

Why Architecture Matters

Same data, same budget, different configuration. Notebook 03_02, 30 random draws from a small search space, one smooth two-dimensional target, 50 epochs each:

- ▶ best draw: validation loss $3.2 \cdot 10^{-6}$
- ▶ hand-picked 3×64 ReLU, 100 epochs: $9.8 \cdot 10^{-6}$
- ▶ worst draw: $4.4 \cdot 10^{-2}$

4 orders of magnitude between the best and the worst draw.

What moved it. Notebook 03_01, one axis at a time: the learning rate changes the loss by $142\times$ over its range, the L_2 penalty by $1.08\times$.

The question

Configuration choice can matter more than training time. Which knobs, and how to search them without trying everything?

Numbers: stored teaching runs of 03_01 and 03_02, seed 0.

The Hyperparameter Space

Architectural hyperparameters:

- ▶ Number of hidden layers
 $L \in \{1, \dots, 5\}$
- ▶ Units per layer $n_\ell \in \{32, 64, \dots, 256\}$
(8 values)
- ▶ Activation: ReLU, tanh, swish

Training hyperparameters:

- ▶ Learning rate η log-uniform on
 $[10^{-4}, 10^{-2}]$ (20-point grid)
- ▶ Batch size $B \in \{16, 32, 64, 128\}$
- ▶ Epochs, regularization, ...

Combinatorial explosion

Even this modest search space:

$$|\mathcal{H}| = 5 \times 8 \times 3 \times 20 \times 4 = 9,600$$

- ▶ Each trial: minutes to hours
- ▶ Full grid: **infeasible**

Manual Tuning

The “grad-student descent” approach:

1. Pick an architecture (intuition / paper)
2. Train, evaluate, adjust one thing
3. Repeat until deadline

Problems:

- ▶ **Doesn't scale** beyond a few hyperparameters
- ▶ **Human bias**: we try what we already know
- ▶ **Not reproducible**: results depend on the researcher
- ▶ Misses interactions between hyperparameters

Key insight

We need **automated, systematic** methods to search the hyperparameter space efficiently.

Automated Search Strategies

Grid Search

Idea: Enumerate all combinations on a regular grid.

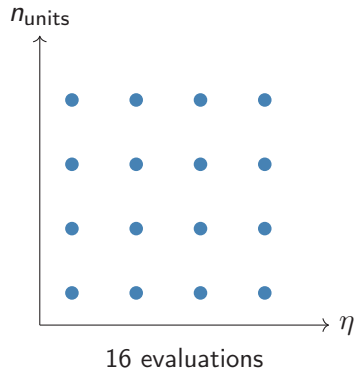
Algorithm

1. Define discrete set for each HP
2. Evaluate *every* combination
3. Return the best

Cost: $\mathcal{O}(k^d)$, exponential in dimension d .

Pros: Simple, embarrassingly parallel.

Cons: Wastes budget on **unimportant** dimensions.



Random Search

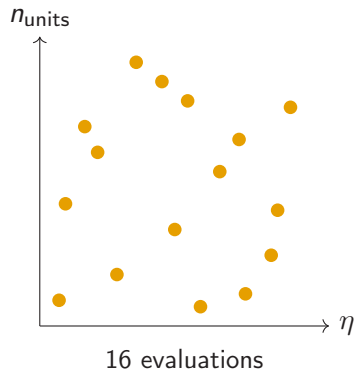
Idea: Sample configurations uniformly at random.

Key result (Bergstra & Bengio, 2012)

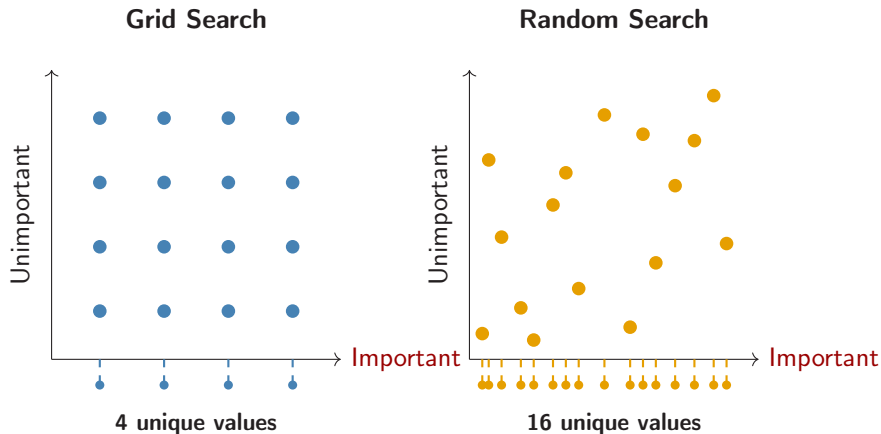
When only a *few* dimensions matter, random search finds good configurations with **fewer trials** than grid search.

Why? Random points explore each dimension *independently*, more unique values per HP.

Cost: $\mathcal{O}(n)$, linear in budget n .



Grid vs. Random: Why Random Wins



When one dimension matters, the grid wastes most evaluations on the unimportant axis; random search samples the important axis more densely.

How Big Is the Edge? A Measurement

Notebook 03_01 scores a k^d grid and k^d random draws on the same surface, $v(h) = (h_1 - h^*)^2 + \delta \sum_{i \geq 2} (h_i - h^*)^2$, 2,000 repetitions. Entry = grid median / random median; > 1 favours random.

	δ	$d=1$	$d=2$	$d=3$	$d=4$	$d=5$	$d=8$
0 (spare axes inert)		0.6	13	140	386	3,305	8,408
	0.01	0.6	2.7	4.9	5.8	6.1	6.5
	0.1	0.7	1.1	1.0	1.2	1.2	1.2
1 (every axis matters)		0.6	0.6	0.6	0.7	0.7	0.8

- ▶ The edge is a statement about **dimension counting**: enormous at $d = 8$, worth little at $d = 2$.
- ▶ It needs the spare axes to be **inert**. When every axis matters, the grid wins: on one axis a grid is a stratified sample.
- ▶ It also needs them to be harmless. A harmful axis (dropout on a clean problem) hands the win to the grid, which always tries the endpoint (03_01, Section 5).

Bayesian Optimization

Idea: Build a *surrogate model* of $f(\text{HP}) \rightarrow \text{loss}$ and use it to choose the next trial *intelligently*.

1. Fit a Gaussian process (GP) to observed trials
2. Compute an **acquisition function** (e.g., Expected Improvement)
3. Evaluate the HP with highest acquisition value
4. Update the GP and repeat

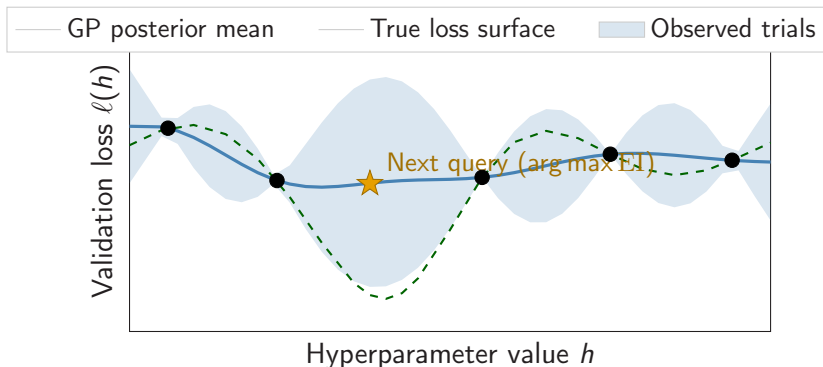
Key advantage: **Sample-efficient**, exploits structure in the response surface.

Expected Improvement

$$\text{EI}(\mathbf{x}) = \mathbb{E}[\max(f^* - f(\mathbf{x}), 0)]$$

- ▶ f^* : best observed value
- ▶ Balances *exploration* (high uncertainty) and *exploitation* (low predicted loss)

Bayesian Optimization: Visual



The GP posterior mean *interpolates* the five observations and is *wide* in unexplored gaps. The Expected-Improvement acquisition function peaks at $h \approx 3.75$, trading off the high posterior variance there against the moderately low predicted mean.

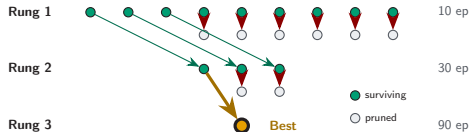
Successive Halving and Hyperband

Idea: start many configurations cheaply, **stop the worst early**, give the budget to the survivors.

Successive halving (SHA), $\eta = 3$

1. Draw n random configurations, train each for b epochs
2. Keep the best $1/\eta$, discard the rest
3. Budget $\times \eta$; *continue training* the survivors
4. Repeat until one is left

Hyperband (Li et al., 2018): several SHA brackets with different (n, b) , a hedge against the unknown correlation between early and late performance.



One bracket, $9 \rightarrow 3 \rightarrow 1$ at $10 \rightarrow 30 \rightarrow 90$ epochs:
 $9 \cdot 10 + 3 \cdot 20 + 1 \cdot 60 = 210$ epochs with
continuation, against 810 for training all nine to the
end. The rung budgets add up only if survivors keep
their weights between rungs.

Method Comparison

Property	Grid	Random	Bayesian	Hyperband
Computational cost	$\mathcal{O}(k^d)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n \log n)$
Sample efficiency	Low	Medium	High	High
Parallelisable	Fully	Fully	Limited	Partially
Handles interactions	No	Partially	Yes	As random
Early stopping	No	No	No	Yes
Implementation	Trivial	Trivial	Moderate	Moderate

Practical advice:

- ▶ **First pass:** random search, several seeds, log-scale on the learning rate. It is the baseline every other method has to beat.
- ▶ **Each trial is expensive and sequential is fine:** Bayesian optimization.
- ▶ **Bad trials show early:** successive halving or Hyperband; count the budget in epochs, and continue survivors rather than restarting them.

Implementing the Search

Random Search and Successive Halving, from scratch

Defining a Search Space

A plain dict, no library required

```
SPACE = {
    "layers":      [1, 2, 3, 4, 5],
    "units":       [32, 64, 96, 128, 160, 192, 224, 256],
    "activation":  ["relu", "tanh", "swish"],
    "lr":          ("log_uniform", 1e-4, 1e-1),
}

def sample_config(rng):
    return {
        "layers":      int(rng.choice(SPACE["layers"])),
        "units":       int(rng.choice(SPACE["units"])),
        "activation":  str(rng.choice(SPACE["activation"])),
        "lr":          float(10 ** rng.uniform(-4, -1)),
    }
```

The search space is a dict, the sampler is one function. The learning-rate range reaches 10^{-1} because 03_01 found the optimum near 10^{-2} ; a range that stops there puts the best region on the boundary.

Random Search from Scratch

30 trials, one function

```
def random_search(n_trials=30, epochs=50, seed=0):  
    rng, records = np.random.default_rng(seed), []  
    for t in range(n_trials):  
        cfg = sample_config(rng)  
        model = build_model(cfg)  
        h = model.fit(X_train, y_train, epochs=epochs,  
                      validation_data=(X_val, y_val), verbose=0)  
        records.append((cfg, min(h.history["val_loss"])))  
    return min(records, key=lambda r: r[1])    # ranked on validation
```

Independent uniform draws; cost exactly $N \cdot R$ epochs, embarrassingly parallel (Bergstra & Bengio, 2012).

Three sets: the search ranks on X_{val} ; X_{test} is touched once, by the winner.

Successive Halving from Scratch

$n_0 = 27$, $\eta = 3$, three rungs, survivors continue

```
def successive_halving(n0=27, eta=3, r0=8, rounds=3, seed=0):
    rng = np.random.default_rng(seed)
    alive = [dict(cfg=c, model=build_model(c), trained=0)
              for c in (sample_config(rng) for _ in range(n0))]
    target = r0
    for k in range(rounds):
        for a in alive:
            # pay only the extra epochs
            a["val"] = fit_epochs(a["model"], target - a["trained"])
            a["trained"] = target
        alive.sort(key=lambda a: a["val"])
        alive = alive[:max(1, len(alive) // eta)] if k < rounds-1 else alive[:1]
        target *= eta
        # 8 -> 24 -> 72 epochs
    return alive[0]
```

Schedule: 27 at 8 epochs $\rightarrow$ 9 continued to 24 $\rightarrow$ 3 continued to 72 $\rightarrow$ 1 winner. Epochs paid:

$27 \cdot 8 + 9 \cdot 16 + 3 \cdot 48 = 504$, against 1500 for 30 random-search trials at 50 epochs. Libraries (KerasTuner, Optuna, Ray Tune) wrap these same loops; the algorithms outlive the APIs.

Example: Genz Function

Approximation

Experiment Setup

Target function: Genz Gaussian on $[0, 1]^2$

$$f(\mathbf{x}) = \exp\left(-\sum_{i=1}^d c_i^2 (x_i - w_i)^2\right)$$

Data:

- ▶ $d = 2$, uniform random samples
- ▶ train 1,000, validation 1,000, test 2,000
- ▶ Loss: MSE; trials ranked on validation loss

Search space:

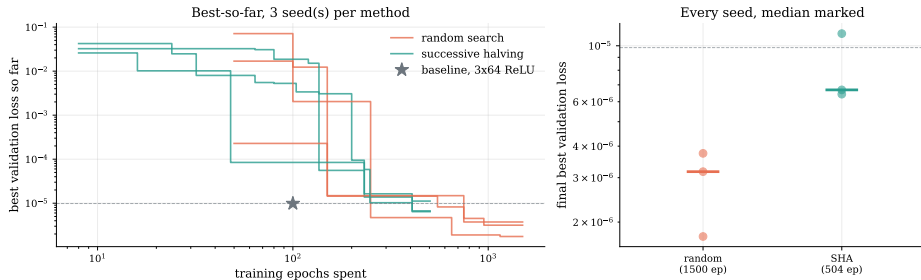
- ▶ Layers: 1–5
- ▶ Units/layer: 32–256 (step 32)
- ▶ Activation: ReLU, tanh, swish
- ▶ LR: 10^{-4} – 10^{-1} (log scale)

Baseline: 3×64 ReLU MLP,
 $\eta = 10^{-3}$, 100 epochs

Budget: random search
 $30 \times 50 = 1500$ epochs; SHA 504
epochs; three seeds each

Search Results on the Genz Gaussian

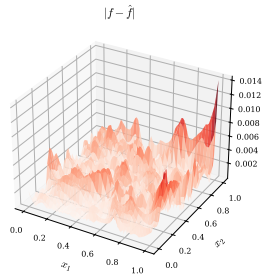
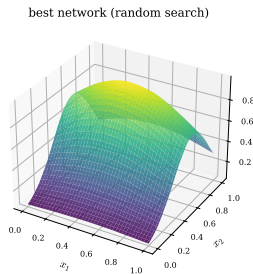
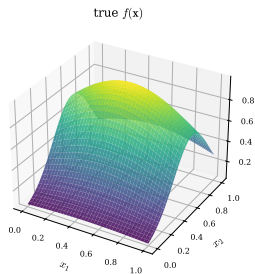
Architecture search on the Genz Gaussian, $d = 2$



Stored teaching run of 03_02, three seeds per method, horizontal axis in epochs paid. Median best validation loss: random search $3.2 \cdot 10^{-6}$ at 1500 epochs, SHA $6.7 \cdot 10^{-6}$ at 504 epochs, baseline $9.8 \cdot 10^{-6}$ at 100. Random search is ahead on all three seeds and the ranges do not overlap (random $1.8 \cdot 10^{-6}$ to $3.7 \cdot 10^{-6}$, SHA $6.4 \cdot 10^{-6}$ to $1.1 \cdot 10^{-5}$), SHA got within a factor of two for a third of the epochs.

Best Model vs. Ground Truth

Winner vs ground truth, test MAE = 0.00158



Stored teaching run of 03_02. Left to right: the true target, the winning configuration retrained from scratch (4L 256-224-128-256 relu lr 5.5e-04), and the absolute error on $[0, 1]^2$. Test MAE 0.00158 against 0.00294 for the hand-picked baseline, on the 2,000 test points the search never saw.

The Same Search on a DEQN

A deep equilibrium net has no labels. A trial trains a policy network on the equilibrium residual and returns an **Euler-equation error**; the search loop does not change.

Model: stochastic Brock–Mirman of Session 2 (02_02): $\alpha = 0.36$, $\beta = 0.99$, $\delta = 0.1$, $\rho = 0.9$, $\sigma = 0.04$; savings rate through a sigmoid, five Gauss–Hermite nodes, states drawn from the 02_02 box. **Score:** mean |relative Euler error| on 4,096 held-out states after 3,000 episodes.

Search space: depth $\{1, 2, 3\}$, width $\{16, 32, 64, 128\}$, relu / tanh, learning rate log-uniform on $[10^{-4}, 10^{-2}]$; 30 random trials.

	mean Euler error
best trial: 3×32 tanh, lr $2.0 \cdot 10^{-3}$	$2.4 \cdot 10^{-4}$
02_02 default: 2×50 relu, lr $3 \cdot 10^{-4}$	$2.1 \cdot 10^{-3}$
worst trial: 1×16 relu, lr $1.6 \cdot 10^{-4}$	$7.9 \cdot 10^{-3}$

median score by level of	spread
width	$3.5 \times$
depth	$2.2 \times$
learning rate, per decade	$1.5 \times$
activation	$1.4 \times$

Stored teaching run of 03_02, Section 7.

Best Practices for NAS

Search space design:

- ▶ Start small, expand if needed
- ▶ Use **log-scale** for learning rate
- ▶ Constrain to reasonable ranges
- ▶ Include the current best as a point

Budget allocation:

- ▶ Count in epochs, not trials
- ▶ Run several seeds; report the range
- ▶ Successive halving: $\eta = 3$, continue survivors

Common pitfalls:

- ▶ Search space too large $\Rightarrow$ wasted budget
- ▶ Ranking on the test set $\Rightarrow$ optimistic numbers; keep a holdout
- ▶ Best region on the boundary of a range $\Rightarrow$ widen it
- ▶ One seed $\Rightarrow$ a lottery ticket, not a result

Before searching

Sweep each axis alone (03_01, Section 3); fix the ones that do not move the error.

Connection to Economics

Where NAS matters in computational economics:

1. **Surrogate models:** Replace expensive simulations (e.g., climate IAMs) with NNs, architecture determines approximation quality.
2. **Policy functions:** Deep equilibrium nets (Session 2) approximate policy functions with a network. The search loop of 03_02, Section 7, runs unchanged with an Euler error as the score.
3. **High-dimensional problems:** As the state grows (Sessions 4 and 5), width and learning rate have to be re-tuned; a random search with a few seeds is the cheap way to do it.
4. **Reproducibility:** Automated search documents *which* architectures were tried, making results more transparent than “we chose 3 layers with 64 units.”

Summary

Key takeaways:

1. Configuration choice moves the error by orders of magnitude; which knob depends on the problem, and the search tells you
2. Random search beats grid search when most axes are inert, and only then
3. Bayesian optimization buys sample efficiency at the price of sequential trials
4. Successive halving spends the epoch budget where it pays; continue survivors, count epochs
5. Rank on validation, report on test, run several seeds; the score can be an Euler error

References:

- ▶ **Elsken, Metzen & Hutter (2019):** *NAS: A Survey*, JMLR 20(55).
- ▶ Bergstra & Bengio (2012). *Random search*. JMLR 13.
- ▶ Li et al. (2018). *Hyperband*. JMLR 18(185).
- ▶ Garnett (2023). *Bayesian optimization*. CUP.
- ▶ Snoek et al. (2012). *Practical Bayesian optimization*. NeurIPS.